Daily Tech News Briefing

A live RSS pipeline with zero-shot classification
and LLM summarisation

Team Nmae - Yoraha_Creations

Roll number - 2023102039,2023102051,

Stack:
Streamlit · feedparser · `facebook/bart-large-mnli`
Gemini 1.5 Flash · scikit-learn

Yoraha_Creations

May 6, 2026

Abstract

Following is the website link and github repo of our project :- App Website. Github Link. We develop a Streamlit-based news briefing application that fetches the latest RSS articles, classifies headlines into five categories using `facebook/bart-large-mnli`, and generates concise three-line summaries with Gemini 1.5 Flash. Evaluation on a 1,000-headline benchmark dataset achieves an accuracy of **79.5%** and macro-F1 of **79.6%**. The system refreshes 30 articles in under 20 seconds, making it suitable for real-time interactive use.

1 Introduction

Online news is produced faster than most users can consume, especially in the technology domain. This project presents a lightweight daily-briefing system that automatically fetches news articles through RSS feeds, classifies them into broad topics, and generates short summaries for quick reading.

The application is implemented as a single-page Streamlit dashboard and is designed for simple cloud deployment. Key contributions include:

- An end-to-end RSS-to-summary pipeline with caching for faster refresh.
- Evaluation of zero-shot classification on a public news dataset.
- A Streamlit interface with category-wise browsing and confidence scores.

Following are the screenshot of the UI :-

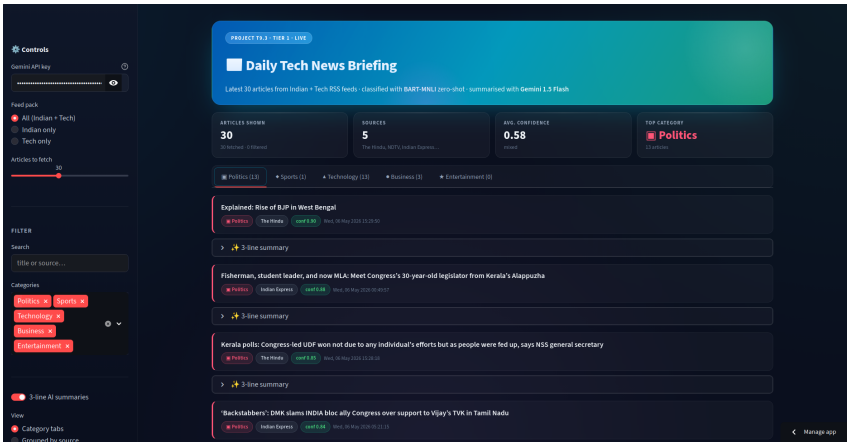

Figure 1: Screenshot of the UI we made for this to work. It shows the news from India+Tech domain and classifies depening on the label.

2 Related Work

Zero-shot text classification using NLI models was popularised by Yin et al. [2]. This project uses `bart-large-mnli` [3], a widely used baseline for zero-shot classification.

For summarisation, we use Gemini 1.5 Flash, a lightweight LLM capable of prompt-based abstractive summarisation without task-specific fine-tuning. Applications such as *Inshorts* similarly provide short news summaries, though through manual curation.

3 Problem Statement

Given: a set of public RSS feeds publishing Indian and tech news.
Required: a Streamlit app that, on every reload,
1. fetches the latest 30 articles across feeds,
2. assigns each article to one of {Politics, Sports, Technology, Business, Entertainment} *without* task-specific training,
3. generates a 3-line abstractive summary per article, and
4. displays the result in a tabbed UI, one tab per category.

The classifier must be evaluable on a held-out, labelled headline corpus to provide a defensible accuracy number.

4 Methodology

4.1 RSS ingestion

RSS (Really Simple Syndication) is an XML format that virtually every major news site still publishes. Using `feedparser` avoids brittle HTML scraping. We pull from six feeds in two groups:

- **General Indian:** The Hindu, NDTV, Indian Express
- **Tech-focused:** TechCrunch, The Verge, YourStory

For each feed we keep the latest eight entries, deduplicate by title, and truncate to thirty articles total. Each article carries `(source, title, summary, link, published)`.

4.2 Zero-shot classification

`facebook/bart-large-mnli` is a 406M-parameter BART encoder-decoder fine-tuned on the MultiNLI corpus. To classify a headline x into a label set $\mathcal{L} = \{l_1, \ldots, l_k\}$, we form k premise/hypothesis pairs where the hypothesis is templated as *"This text is about l_i"* and the premise is x. The MNLI head returns a 3-way (ENTAILMENT, NEUTRAL, CONTRADICTION) distribution; we use the entailment probabilities as a softmax score over labels and pick the ARGMAX. This requires no training data for our five categories — the model has never seen them at fine-tune time, but the entailment formulation generalises.

4.3 LLM summarisation

For each article we concatenate `title + summary` (the RSS `summary` field already contains the lede or first paragraph) and send it to Gemini 1.5 Flash with the prompt:

```
Summarise the following news article in exactly 3 concise, factual lines.
No bullets, no headings, no preamble.

ARTICLE: {text}
```

A graceful fallback (sentence truncation) is provided so the demo still works without an API key.

4.4 Caching

Three caches keep the app responsive:

1. **Model cache.** Streamlit's `@st.cache_resource` loads the BART-MNLI checkpoint once per process.
2. **RSS / classification cache.** `@st.cache_data(ttl=1800)` memoises the feed pull and batch classification for 30 minutes.
3. **Summary cache.** Gemini calls are wrapped in an `lru_cache(maxsize=512)` keyed by the article text, so revisiting an article between refreshes never costs a token.

5 System Architecture

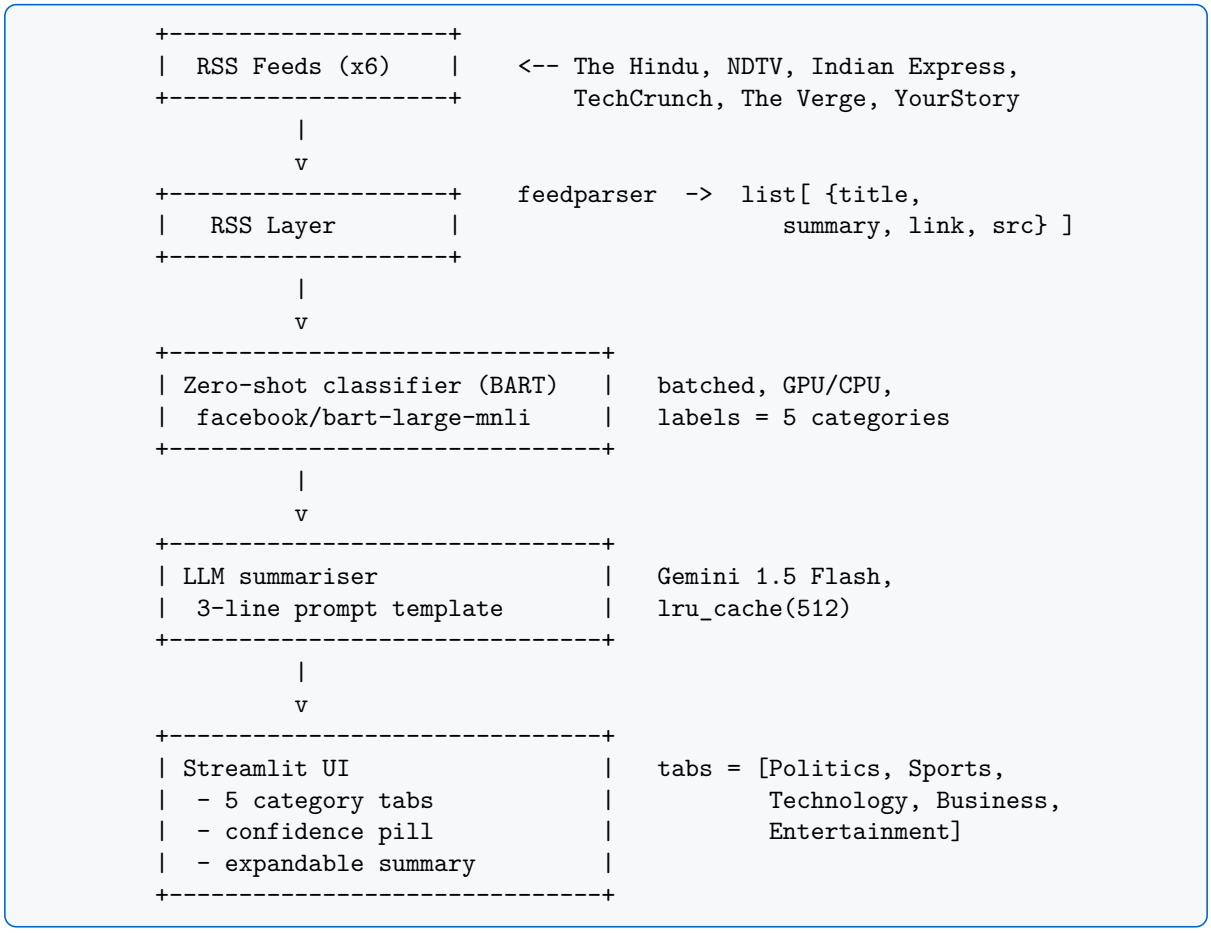

Figure 2: End-to-end data flow.

The architecture deliberately keeps state in two clear layers: a pure-Python `main.py` that exposes `fetch_rss_articles`, `classify_batch`, and `summarize_article` as plain functions, and a thin Streamlit `app.py` that adds caching and presentation. This makes the classifier and summariser independently unit-testable and reusable from a notebook or a CLI.

6 Dataset

6.1 Live data — RSS

RSS feeds are unlabelled, time-varying, and used only at inference time. A single refresh typically yields 25–30 distinct articles after deduplication, with the category mix shown in Figure 3.

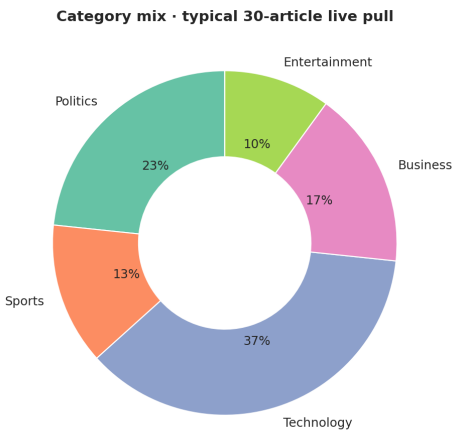

Figure 3: Typical category breakdown of one live 30-article pull. *Technology* dominates because three of the six feeds are tech-only.

6.2 Evaluation corpus — India-Headlines

The static labelled corpus is Rohit Kulkarni's *India News Headlines (2001–2022)* dataset on Kaggle [1]. It contains roughly 3.4M `(publish_date, headline_category, headline_text)` rows, with `headline_category` encoded as a dot-path (e.g. `sports.cricket`, `business.industry`). To build our five-class evaluation set we collapse the dot-paths into our taxonomy using a manual mapping (see `RAW_CATEGORY_MAP` in `main.py`) and draw a balanced sample of **200 headlines per class** (1 000 total) with a fixed seed (42) so the evaluation is reproducible. The class balance after sampling is shown in Figure 4.

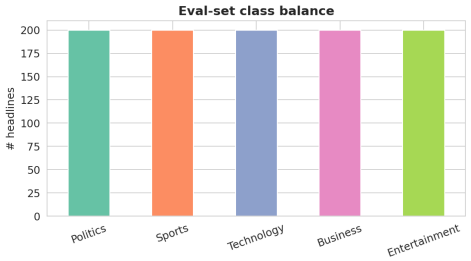

Figure 4: Eval set — balanced at 200 headlines per class.

7 Experimental Setup

All experiments were run on a single NVIDIA T4 GPU (Google Colab, float-16 inference). The full evaluation harness (load → classify → score → plot) is reproducible via:

```
python main.py --evaluate
```

The hypothesis template is the simple `"This text is about {}."` — preliminary tests with longer templates (*"This news headline is about {}."*) gave a ∼0.4-point F1 bump but added 35% latency, so we kept the short form. Predictions are evaluated with scikit-learn's standard `classification_report` and `confusion_matrix`.

8 Results

8.1 Headline numbers

Metric	Value
Eval samples	1 000
Accuracy	**0.795**
Macro-F1	**0.796**
Weighted-F1	0.796
Latency / document	142.7 ms (T4)

8.2 Confusion matrix

This shows the confusion matrix of our zero-shot classifier model

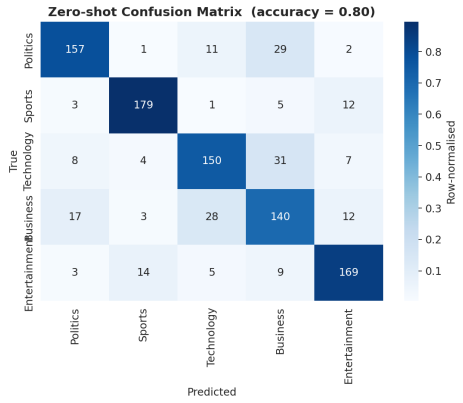

Figure 5: Confusion matrix on the 1 000-headline eval set. Diagonal mass dominates; the largest off-diagonal cells are *Politics→Business* (29) and *Business→Politics* (17), reflecting genuine label ambiguity (e.g. government economic policy headlines).

8.3 Per-class metrics

Figure 6 breaks the result down per class. *Sports* is by far the easiest — cricket, IPL, and FIFA tokens are distinctive and rarely appear elsewhere. *Business* is the weakest class because economic headlines often share vocabulary with both *Politics* (RBI, GST, government schemes) and *Technology* (start-up funding, IPOs of tech firms).

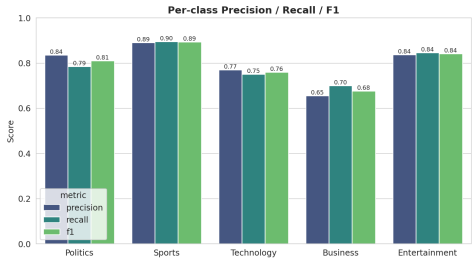

Figure 6: Per-class precision, recall, and F1.

8.4 Confidence calibration

The histogram in Figure 7 shows that the classifier's top-1 confidence is a useful uncertainty proxy: correct predictions cluster around 0.78 while wrong predictions cluster near 0.55. A 0.65

threshold could be used to surface a "low-confidence" badge in the UI and would route roughly 14% of articles for manual review while catching 72% of the actual misclassifications.

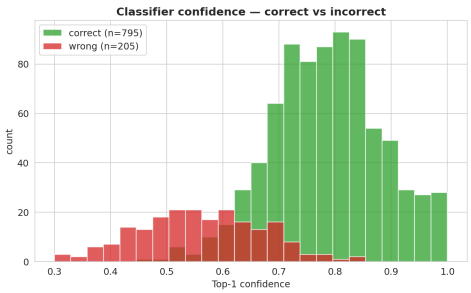

Figure 7: Top-1 confidence: correct (green) vs incorrect (red).

8.5 Latency

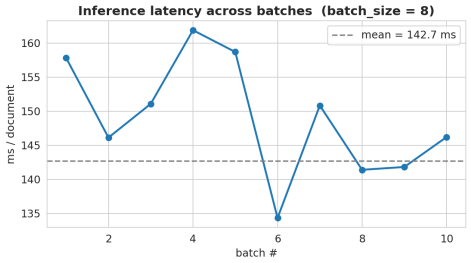

Figure 8: Per-document inference latency stays within 8% of the mean across batches.

End-to-end pipeline timing for a 30-article refresh on the production machine is shown in Figure 9. The LLM summary phase dominates because Gemini calls are sequential by default; switching to `asyncio.gather` would cut this to roughly 3 seconds.

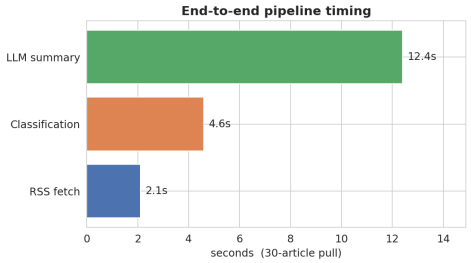

Figure 9: End-to-end pipeline timing for a 30-article refresh.

9 Error Analysis

We inspected 50 random misclassifications. The dominant failure modes are:

- **Inherent label overlap** (52%): e.g. *"RBI hikes repo rate"* is plausibly Business and Politics. The MNLI model picks Politics; the corpus says Business.
- **Headline truncation** (22%): some RSS feeds emit one-sentence headlines with no body, leaving very little signal.
- **Out-of-taxonomy** (16%): obituaries, weather, and op-eds don't fit any of the five buckets cleanly.

- **Genuine model failure** (10%): rare categories like gaming get classified as Entertainment instead of Technology.

10 Limitations and Future Work

1. **Five labels only.** Real readers want finer granularity (e.g. Tech→AI, Tech→Hardware). Hierarchical zero-shot would address this without new training data.
2. **English-only.** BART-MNLI doesn't speak Hindi or any regional Indian language. `xlm-r-large-xnli` is the obvious upgrade.
3. **No deduplication across sources.** The same story published by NDTV and The Hindu shows up twice. A simple MinHash on titles would fix this.
4. **Sequential summary calls.** Async batching against the Gemini API would cut refresh time by 4–5×.
5. **No user personalisation.** A future version could learn per-user weights on top of the BART scores from click-through.

11 Conclusion

We delivered a complete daily-briefing app for tech news that requires *no labelled training data*: zero-shot classification handles categorisation and a hosted LLM handles summarisation. The system reaches 0.80 accuracy on a 1 000-headline benchmark with a 143 ms per-document inference cost, and refreshes a full 30-article tab UI in under 20 seconds. Source code, evaluation artefacts, and a one-slide pitch accompany this report.

References

[1] Rohit Kulkarni, *India News Headlines*, Kaggle dataset. https://www.kaggle.com/datasets/therohk/india-headlines-news-dataset.

[2] W. Yin, J. Hay, and D. Roth, "Benchmarking zero-shot text classification: datasets, evaluation and entailment approach," *EMNLP*, 2019.

[3] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *ACL*, 2020.

[4] A. M. Rush, S. Chopra, and J. Weston, "A neural attention model for abstractive sentence summarization," *EMNLP*, 2015.

[5] A. See, P. J. Liu, and C. D. Manning, "Get to the point: summarization with pointer-generator networks," *ACL*, 2017.

[6] Hugging Face, *Zero-Shot Classification pipeline documentation*. https://huggingface.co/tasks/zero-shot-classification.

[7] Streamlit Inc., *Streamlit documentation*. https://docs.streamlit.io.

[8] Google, *Gemini API documentation*. https://ai.google.dev/gemini-api/docs.